

L6: Sorting and Symbolic Processing

CS1101S: Programming Methodology

Sanka Rasnayaka

September 20, 2023

Outline

- Binary Search and Binary Search Trees (2.3.3)
- Sorting
- Symbolic Processing (2.3.2)
- Designing Data Structures (2.4)

Announcements

- Next week is **Recess Week**
- **Midterm Assessment** on Wednesday of Week 7 (4 Oct), 10am
- This Friday's brief: **Guest speaker** from Jane Street
 - Speaker: **Keloysius Mak**
 - Title of talk: ***Why OCaml***

Update on Contests

- Contest “**Games of Tones**” opened on Monday, due 28 Sep
- **Voting** for “**The Choreographer**” ended yesterday

Outline

- Binary Search and Binary Search Trees (2.3.3)
- Sorting
- Symbolic Processing (2.3.2)
- Designing Data Structures (2.4)

Game: Guess the Secret Number

- I have a **secret number**, an integer in the range **[1, 100]**
- You make a **guess** and I tell you whether your guess is “**correct**”, “**too low**”, or “**too high**”
- Your goal is to **minimize** the **number of guesses**
- What is your **strategy**?
 - Guess the “**middle**” number
 - If “**too low**”, continue to find in the “**right**” half
 - If “**too high**”, continue to find in the “**left**” half
 - **Binary Search!!!**

Guess the Secret Number using Binary Search

```
const N = 100;
...
function guess_secret_num(start, end) {
  if (start === end) {
    return start;
  } else {
    const guess = math_floor((start + end) / 2);
    const check = check_guess(guess);
    return check === "correct"
      ? guess
      : check === "too low"
        ? guess_secret_num(guess + 1, end)    // when "too low"
        : guess_secret_num(start, guess - 1); // when "too high"
  }
}
guess_secret_num(1, N);
```

[Show in
Playground](#)

Observations

- **Run time**

- At each step (wrong guess), we **cut the search space in half**
- If problem size is $N = 2^k$, we get to size 1 after k steps
- Run time: **$O(\log_2 N)$**
 - Equivalent to **$O(\log N)$**

- **Why does this work?**

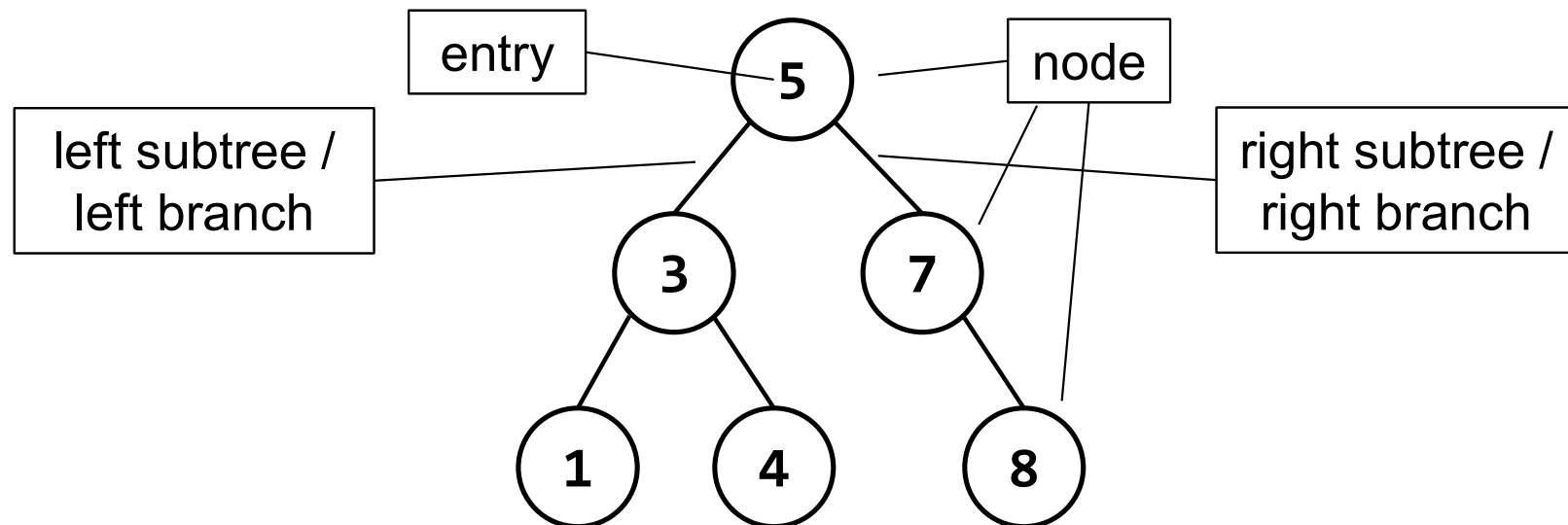
- We need to assume that
 - **all numbers** to the “**left**” of the **secret number** are “**too low**”, and
 - **all numbers** to the “**right**” of the **secret number** are “**too high**”

Binary Search for Entries

- **The problem**
 - Check if an **entry** is included in a **collection of entries**
- **Property of entries**
 - A **total order** exists — two entries can be compared
 - They are “equal”, or one is either “smaller” or “larger” than the other
 - Examples: Numbers and Strings have such property
- **Efficiency**
 - Need to reach each “middle” entry in $O(1)$ time
 - May need a special data structure

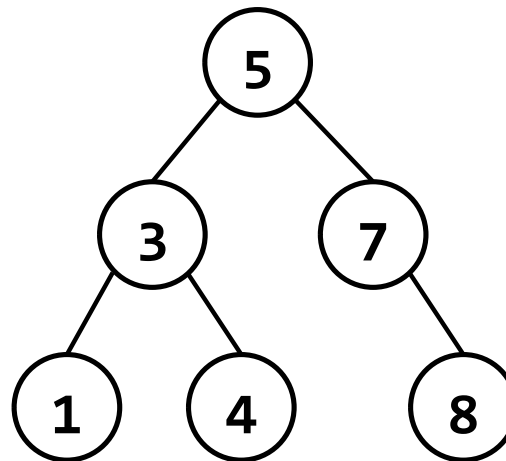
Binary Trees

- A **binary tree** is the **empty tree**, or it has
 - an **entry** (which is the data item)
 - a **left branch** or **left subtree** (which is a binary tree)
 - a **right branch** or **right subtree** (which is a binary tree)



Binary Search Trees (BST)

- A **binary search tree (BST)** is a **binary tree** where
 - all entries in the **left subtree** are **smaller than** the entry, and
 - all entries in the **right subtree** are **larger than** the entry



- A **BST** is an **abstraction** for **binary search**

Mission “Search and Rescue” (opens today)

- **Given:** A binary search tree (BST) of Strings
- **Problem 1:** Check if a given String occurs in BST
 - Use BST property to efficiently do so
- **Problem 2:** Add a new string to BST
 - Use BST property to efficiently do so
 - Preserve BST property

Outline

- Binary Search and Binary Search Trees (2.3.3)
- Sorting
 - The Problem of Sorting
 - Insertion Sort
 - Selection Sort
 - Merge Sort
- Symbolic Processing (2.3.2)
- Designing Data Structures (2.4)

The Problem of Sorting

- **Given:**
 - A list xs of elements from a given universe X , and a ***total order*** on X
- **Wanted:**
 - A permutation of xs such that each element is **greater than or equal** to the previous one, with respect to the total order
- **Comparisons only**
 - The only allowed operations on the elements are comparisons, such as $<$, $>$, $<=$, $>=$, $==$ or $!=$

How to Sort a List with n Elements?

- **Strategy:**
 - Wishful thinking!
 - Imagine we can sort lists with m elements, where $m < n$
- **Approach A**
 - $m = n - 1$
- **Approach B**
 - $m = n / 2$

Approach A: Insertion Sort

- **Idea:**
 - **Sort the tail** of the given list using wishful thinking!
 - **Insert the head** in the right place

- **Implementation:**

```
function insertion_sort(xs) {  
    return is_null(xs)  
        ? xs  
        : insert(head(xs), insertion_sort(tail(xs)));  
}
```


Insertion Sort

```
function insert(x, xs) {  
  return is_null(xs)  
    ? list(x)  
    : x <= head(xs)  
    ? pair(x, xs)  
    : pair(head(xs), insert(x, tail(xs)));  
}  
  
function insertion_sort(xs) {  
  return is_null(xs)  
    ? xs  
    : insert(head(xs), insertion_sort(tail(xs)));  
}
```

- Order of growth in time?

[Show in
Playground](#)

Approach A: Selection Sort

- **Idea:**

- Find the **smallest element x** and **remove** it from the list
- **Sort the remaining list**, and **put x in front**

- **Implementation:**

```
function selection_sort(xs) {  
  if (is_null(xs)) {  
    return xs;  
  } else {  
    const x = smallest(xs);  
    return pair(x, selection_sort(remove(x, xs)));  
  }  
}
```

Selection Sort

```
// find smallest element of non-empty list xs
function smallest(xs) {
    return accumulate((x, y) => x < y ? x : y,
                      head(xs), tail(xs));
}
function selection_sort(xs) {
    if (is_null(xs)) {
        return xs;
    } else {
        const x = smallest(xs);
        return pair(x, selection_sort(remove(x, xs)));
    }
}
```

- Order of growth in time?

[Show in
Playground](#)

Recall: How to Sort a List with n Elements?

- **Strategy:**
 - Wishful thinking!
 - Imagine we can sort lists with m elements, where $m < n$
- **Approach A**
 - $m = n - 1$
- **Approach B**
 - $m = n / 2$
 - **Idea: Split** the list **in half**, sort each half using wishful thinking, **merge** the sorted lists together

Approach B: Merge Sort

- **Idea:**

- **Split** the list **in half**, sort each half using wishful thinking
- **Merge** the sorted lists together

- **Implementation:**

```
function merge_sort(xs) {  
  if (is_null(xs) || is_null(tail(xs))) {  
    return xs;  
  } else {  
    const mid = middle(length(xs));  
    return merge(merge_sort(take(xs, mid)),  
                 merge_sort(drop(xs, mid)));  
  }  
}
```

Merge Sort (Function merge)

```

function merge(xs, ys) {
  if (is_null(xs)) {
    return ys;
  } else if (is_null(ys)) {
    return xs;
  } else {
    const x = head(xs);
    const y = head(ys);
    return x < y
      ? pair(x, merge(tail(xs), ys))
      : pair(y, merge(xs, tail(ys)));
  }
}

```

xs	ys	result												
<table><tr><td>2</td><td>4</td><td>5</td></tr></table>	2	4	5	<table><tr><td>3</td><td>7</td><td>8</td></tr></table>	3	7	8	<table><tr><td></td><td></td><td></td><td></td><td></td><td></td></tr></table>						
2	4	5												
3	7	8												
<table><tr><td>2</td><td>4</td><td>5</td></tr></table>	2	4	5	<table><tr><td>3</td><td>7</td><td>8</td></tr></table>	3	7	8	<table><tr><td>2</td><td></td><td></td><td></td><td></td><td></td></tr></table>	2					
2	4	5												
3	7	8												
2														
<table><tr><td>2</td><td>4</td><td>5</td></tr></table>	2	4	5	<table><tr><td>3</td><td>7</td><td>8</td></tr></table>	3	7	8	<table><tr><td>2</td><td>3</td><td></td><td></td><td></td><td></td></tr></table>	2	3				
2	4	5												
3	7	8												
2	3													
<table><tr><td>2</td><td>4</td><td>5</td></tr></table>	2	4	5	<table><tr><td>3</td><td>7</td><td>8</td></tr></table>	3	7	8	<table><tr><td>2</td><td>3</td><td>4</td><td></td><td></td><td></td></tr></table>	2	3	4			
2	4	5												
3	7	8												
2	3	4												
<table><tr><td>2</td><td>4</td><td>5</td></tr></table>	2	4	5	<table><tr><td>3</td><td>7</td><td>8</td></tr></table>	3	7	8	<table><tr><td>2</td><td>3</td><td>4</td><td>5</td><td></td><td></td></tr></table>	2	3	4	5		
2	4	5												
3	7	8												
2	3	4	5											
<table><tr><td>2</td><td>4</td><td>5</td></tr></table>	2	4	5	<table><tr><td>3</td><td>7</td><td>8</td></tr></table>	3	7	8	<table><tr><td>2</td><td>3</td><td>4</td><td>5</td><td>7</td><td>8</td></tr></table>	2	3	4	5	7	8
2	4	5												
3	7	8												
2	3	4	5	7	8									

[Show in
Playground](#)

Merge Sort (Other Helper Functions)

// half, rounded downwards

```
function middle(n) {  
    return math_floor(n / 2);  
}
```

// put the first n elements of xs into a list

```
function take(xs, n) {  
    // ???  
}
```

// drop the first n elements from the list

// and return the rest

```
function drop(xs, n) {  
    // ???  
}
```

[Show in
Playground](#)

Merge Sort

- Order of growth in time?
 - We will discuss this in **Reflection R6**

Approach B: QuickSort

- **Question:** Is there another way of “splitting” and “combining”?
- **Answer:** Yes! Details in Mission “**Sorting Things Out**”

Outline

- Binary Search and Binary Search Trees (2.3.3)
- Sorting
- Symbolic Processing (2.3.2)
 - Symbolic Evaluation
 - Symbolic Differentiation
- Designing Data Structures (2.4)

Representing Functions: Directly

- Our first approach is to represent functions ***directly*** in Source using **function declarations** and **lambda expressions**
- **Example:**

$$f(x) = x^2 + 1$$

```
function my_fun(x) {  
    return x * x + 1;  
}
```

```
my_fun(7); // returns 50
```

Numerical Differentiation

```
// numerical differentiation; simplest method  
function deriv_numeric(f) {  
    const dx = 0.0001;  
    return x => (f(x + dx) - f(x)) / dx;  
}
```

// Example use:

```
const f = x => x * x + x + 4;  
const f_prime = deriv_numeric(f);
```

```
f(3);           // returns 16  
f_prime(3);     // returns 7.0001000000027487
```

[Show in
Playground](#)

Symbolic Evaluation

- Now, we represent functions / expressions with **data structures**

- **Example expression:**

```
// my_exp represents  $x * x + x + 4$   
const my_exp = make_sum(make_product("x", "x"),  
                        make_sum("x", 4));
```

- **Symbolic evaluation:**

```
eval_symbolic(my_exp, "x", 3);  
// should return 16
```

Definition of eval_symbolic

```
function eval_symbolic(exp, name, val) {  
  return is_number(exp)  
    ? exp  
    : is_variable(exp)  
    ? (is_same_variable(exp, name) ? val : NaN)  
    : is_sum(exp)  
    ? eval_symbolic(addend(exp), name, val) +  
      eval_symbolic(augend(exp), name, val)  
    : is_product(exp)  
    ? eval_symbolic(multiplier(exp), name, val) *  
      eval_symbolic(multiplicand(exp), name, val)  
    : error(exp, "unknown expression type");  
}
```

[Show in
Playground](#)

Symbolic Differentiation

- **Symbolic representation:**

```
// my_exp represents x * x + x + 4  
const my_exp = make_sum(make_product("x", "x"),  
                        make_sum("x", 4));
```

- **Symbolic differentiation:**

```
deriv_symbolic(my_exp, "x");  
// should return make_sum(make_product("x", 2), 1)  
  
eval_symbolic(deriv_symbolic(my_exp, "x"), "x", 3);  
// should return 7
```

$$f(x) = x^2 + x + 4$$
$$f'(x) = 2x + 1$$

Symbolic Differentiation

- Rules of differentiation

$$\frac{dc}{dx} = 0 \text{ for } c \text{ a constant or a variable different from } x$$

$$\frac{dx}{dx} = 1$$

$$\frac{d(u + v)}{dx} = \frac{du}{dx} + \frac{dv}{dx}$$

$$\frac{d(uv)}{dx} = u \left(\frac{dv}{dx} \right) + v \left(\frac{du}{dx} \right)$$

Definition of deriv_symbolic

```
function deriv_symbolic(exp, x) {  
  return is_number(exp)  
    ? 0  
    : is_variable(exp)  
    ? (is_same_variable(exp, x) ? 1 : 0)  
    : is_sum(exp)  
    ? make_sum(deriv_symbolic(addend(exp), x),  
               deriv_symbolic(augend(exp), x))  
    : is_product(exp)  
    ? make_sum(make_product(multiplier(exp),  
                           deriv_symbolic(multiplicand(exp), x)),  
               make_product(  
                 deriv_symbolic(multiplier(exp), x),  
                 multiplicand(exp)))  
    : error(exp, "unknown expression type");  
}
```

[Show in
Playground](#)

Implementation of Symbolic Representation

- **Constructors:**

```
function make_sum(a1, a2) {  
    return list("+", a1, a2);  
}
```

```
function make_product(m1, m2) {  
    return list("*", m1, m2);  
}
```

Implementation of Symbolic Representation

- **Accessors / Selectors:**

```
function addend(s) {  
    return head(tail(s));  
}  
  
function augend(s) {  
    return head(tail(tail(s)));  
}  
  
function multiplier(s) {  
    return head(tail(s));  
}  
  
function multiplicand(s) {  
    return head(tail(tail(s)));  
}
```

Implementation of Symbolic Representation

- **Predicates:**

```
function is_variable(x) {  
    return is_string(x);  
}  
  
function is_same_variable(v1, v2) {  
    return is_variable(v1) && is_variable(v2) && v1 === v2;  
}  
  
function is_sum(x) {  
    return is_pair(x) && head(x) === "+";  
}  
  
function is_product(x) {  
    return is_pair(x) && head(x) === "*";  
}
```

Revisiting Example

- Symbolic representation:**

```
// my_exp represents x * x + x + 4
const my_exp = make_sum(make_product("x", "x"),
                        make_sum("x", 4));
```

- Symbolic differentiation:**

```
deriv_symbolic(my_exp, "x");
// should return make_sum(make_product("x", 2), 1)
```

```
// but instead returns a complicated expression
// equivalent to x * 1 + 1 * x + 1 + 0
```

$$f(x) = x^2 + x + 4$$

$$f'(x) = 2x + 1$$

make_sum with Expression Simplification

```
function make_sum(a1, a2) {  
    return number_equal(a1, 0)  
        ? a2  
        : number_equal(a2, 0)  
            ? a1  
            : is_number(a1) && is_number(a2)  
                ? a1 + a2  
                : is_variable(a1) && is_variable(a2)  
                    && is_same_variable(a1, a2)  
                        ? list("*", 2, a1)  
                        : list("+", a1, a2);  
}
```

[Show in
Playground](#)

make_product with Expression Simplification

```
function make_product(m1, m2) {  
    return number_equal(m1, 0) ||  
           number_equal(m2, 0))  
    ? 0  
    : number_equal(m1, 1)  
    ? m2  
    : number_equal(m2, 1)  
    ? m1  
    : is_number(m1) && is_number(m2)  
    ? m1 * m2  
    : list("?", m1, m2);  
}
```

[Show in
Playground](#)

Revisiting Example, Again

- **Symbolic representation:**

```
// my_exp represents x * x + x + 4  
const my_exp = make_sum(make_product("x", "x"),  
                        make_sum("x", 4));
```

- **Symbolic differentiation:**

```
deriv_symbolic(my_exp, "x");  
// should return make_sum(make_product("x", 2), 1)
```

```
// now returns an expression  
// equivalent to 2 * x + 1
```

$$f(x) = x^2 + x + 4$$
$$f'(x) = 2x + 1$$

Outline

- Binary Search and Binary Search Trees (2.3.3)
- Sorting
- Symbolic Processing (2.3.2)
- Designing Data Structures (2.4)
 - Specification
 - Implementation

Example: Symbolic Differentiation

- Example **constructor**:

```
function make_sum(a1, a2) {  
    return list("+", a1, a2);  
}
```

- Example **accessor / selector**:

```
function addend(s) {  
    return head(tail(s));  
}
```

- Example **predicate**:

```
function is_sum(x) {  
    return is_pair(x) && head(x) === "+";  
}
```

Example: Symbolic Differentiation

- These functions
 - Provide an **abstraction** of the data structure
 - “**Shield**” the rest of the program from the **implementation** of the data structure
 - For example, `eval_symbolic` and `deriv_symbolic` do not need to know the actual implementation

Process of Designing Data Structures

- **Specification:** describes ***what*** is done (contract)
- **Implementation:** describes ***how*** it is done (work)

Set: Specification

- A **set** is an *unordered* collection of objects
 - (here, let us consider only numbers as objects)
- **Constructors:**
 - `make_empty_set()`, `add(e, S)`, `remove(e, S)`
- **Predicates:**
 - `find(e, S)`
- **Contract:**
 - `find(e, add(e, S))` returns true
 - `find(e, remove(e, S))` returns false

Set: Implementation (Version 1)

- **Idea:** Represent a set by an **unordered list**
 - Consequence for `add(e, S)` ?
 - Consequence for `remove(e, S)` ?
 - Consequence for `find(e, S)` ?

Set: Implementation (Version 2)

- **Idea:** Represent a set by an **ordered list**
 - Consequence for $\text{add}(e, S)$?
 - Consequence for $\text{remove}(e, S)$?
 - Consequence for $\text{find}(e, S)$?

Set: Implementation (Version 3)

- **Idea:** Represent a set by a **binary search tree**
 - Consequence for $\text{add}(e, S)$?
 - Consequence for $\text{remove}(e, S)$?
 - Consequence for $\text{find}(e, S)$?

Which implementation of Set is the best?

- Several reasonable choices
- Some operations are **fast** in one implementation and **slow** in another one
- Often there is a **trade-off**
- **Best** representation depends on which operations are **most frequent**

Summary

- **Binary search and binary search trees (BST)**
- **Sorting**
 - Approach A: **insertion sort, selection sort**
 - Approach B: **merge sort, quicksort**
- **Symbolic processing**
 - Evaluation & differentiation
- **Data structure design**
 - **Specification vs implementation**
 - Consider implementation **trade-offs**

Concluding Unit 2

- Covered: textbook chapter 2
- Data abstraction
- Data structures: pair, list, tree
- Data processing, list processing, tree processing, sorting
- **Unit 3** – State & Mutable Data starts from L7
- **Special Event** – Guest Lecture, B6